SkillSync AI

Project Documentation

Intelligent Employee Skill Gap Analysis & Training Recommendation System

Discover Your Potential. Bridge Your Skill Gaps. Grow Smarter.

1. Overview

SkillSync AI is a full-stack web application that helps organizations find the gap between the skills their employees have and the skills their job roles require, and then recommends training to close those gaps. It replaces manual spreadsheets with a single platform where HR, managers and employees see the same, up-to-date picture of workforce skills.

2. Objectives

- Record employee skills and proficiency levels (scale 1 to 5).
- Define the skills and levels each job role requires.
- Calculate the gap between current and required levels, and a readiness score for each employee.
- Recommend relevant courses that target the largest gaps.
- Give each type of user a dashboard suited to their responsibilities.
- Keep accounts secure with hashed passwords, token-based sessions and role-based access.

3. Users and Roles

Role	What they can do
HR Administrator	Full access: overview analytics, employee directory, skill analysis, training courses, job roles, reports and settings.
Manager	Overview, employee directory, skill analysis, training courses and reports. Cannot manage job roles.
Employee	Own overview, own skill analysis, recommended courses and settings. Cannot see other employees.

Access is enforced in two places: the frontend hides pages a role should not see, and the backend rejects API calls the role is not allowed to make (HTTP 401 for a missing or expired token, 403 for a forbidden role).

4. System Architecture

The system has four parts that communicate over HTTP:

1. **React frontend** (Vite): the user interface, running on port 5173.
2. **Spring Boot backend** (Java 17): the main REST API and authentication, running on port 8080.
3. **MySQL database**: hosted online on Aiven (free tier), accessed by the backend over SSL.
4. **Python recommendation service** (FastAPI): a supporting service that ranks courses using TF-IDF cosine similarity, running on port 8000.

Flow: the browser talks to the Spring Boot API; Spring Boot reads and writes MySQL; the Python service is called for course recommendations. If the backend is not running, the frontend falls back to a local demo mode so the interface still works.

5. Technology Stack

Layer	Technology
Frontend	React 18, Vite, Recharts (charts), Lucide (icons), plain CSS
Backend	Java 17, Spring Boot 3.3.5, Spring Data JPA, Bean Validation
Security	BCrypt password hashing (spring-security-crypto), JWT (jjwt, HS256)
Database	MySQL 8 on Aiven, SSL required
Recommender	Python, FastAPI, TF-IDF and cosine similarity
Build tools	npm, Maven (wrapper included, no install needed)

6. Project Structure

- `frontend/`: React app. `src/App.jsx` (dashboard and pages), `src/Login.jsx` (sign in, register, forgot password), `src/auth.js` (API calls and demo fallback), `src/styles.css`.
- `backend/`: Spring Boot app. `controller/` (Auth, Employee, SkillAnalysis, Health), `model/User.java`, `repo/UserRepository.java`, `security/` (JwtService, AuthInterceptor), `config/` (WebConfig, DemoDataSeeder), `resources/application*.properties`.
- `database/`: `schema.sql` (local MySQL) and `schema_cloud.sql` (safe to run repeatedly on a hosted database).
- `python-service/`: `app.py` recommendation service.
- `README.md` and `CLOUD_DATABASE.md`: setup notes.

7. Database Design

The schema contains 12 tables:

Table	Purpose
departments	Company departments.
users	Login accounts: name, email, password hash, role (ADMIN, MANAGER, EMPLOYEE).
employees	Employee profile linked to a user, job role and manager.
skills	Skill catalogue with category (for example Java, SQL, Communication).
employee_skills	Each employee's proficiency (1 to 5) per skill.
job_roles	Job titles and descriptions.
role_required_skills	Required level (1 to 5) per skill for each job role.
courses	Training courses with difficulty, duration and provider.
course_skills	Which skills a course develops and the target level.
enrollments	Employee course enrollment, status and progress.
skill_assessments	History of assessed levels and who assessed them.
recommendations	Recommended courses per employee with relevance score and reason.

Key relationships: a user has one employee profile; an employee has many skills; a job role has many required skills; a course targets many skills.

8. Core Logic

Skill gap. For each skill required by an employee's job role, gap = required level minus current level. A positive value means a gap.

Readiness score. The average of min(current / required, 1) across all required skills, shown as a percentage. An employee who meets every requirement scores 100%.

Recommendations. Courses are ranked by how well their skills match the employee's largest gaps. The Python service uses TF-IDF vectors and cosine similarity between the gap description and course descriptions.

9. Authentication and Security

- Passwords are hashed with BCrypt and never stored in plain text.
- On login or registration the backend returns a JWT (valid 12 hours by default) containing the user's email and role.
- The frontend sends the token in the `Authorization: Bearer` header, and an interceptor validates it on protected routes.
- `/api/employees` is limited to HR and Manager roles.
- Public sign-up cannot create HR accounts unless `ALLOW_ADMIN_SIGNUP=true`.
- The forgot-password endpoint returns the same message whether or not the account exists, which prevents email enumeration.
- CORS only allows the configured frontend origin.

10. REST API Reference

Method and path	Access	Description
POST /api/auth/register	Public	Create an account and return a token.
POST /api/auth/login	Public	Sign in and return a token.
POST /api/auth/forgot	Public	Request a password reset link (email sending not yet connected).
GET /api/auth/me	Signed in	Get the current user.
PUT /api/auth/me	Signed in	Update the display name.
PUT /api/auth/password	Signed in	Change password (requires current password).
GET /api/employees	HR, Manager	List employees (currently sample data).
POST /api/skill-analysis/{id}	Signed in	Skill gap and readiness for an employee (currently sample data).
GET /api/health	Public	Service status.
POST /recommend (port 8000)	Internal	Ranked course recommendations from the Python service.

Errors are returned as JSON with a `message` field.

11. Frontend Pages

- **Login / Register / Forgot password:** role picker, show-password toggle, validation and one-click demo accounts.
- **Overview:** metric cards, workforce skill chart, department readiness, employees needing attention and recent activity.
- **Employees:** searchable, filterable directory and an add-employee form.
- **Skill Analysis:** radar chart of current versus required skills, gap breakdown and an AI insight.
- **Training Courses:** recommended courses with match percentage and enrollment.
- **Job Roles:** role cards and requirements.
- **Reports & Analytics:** key figures and skill development trend.
- **Settings:** edit profile, change password, notification preferences. A notification dropdown and logout are available throughout.

12. Setup and Running

Requirements: Java 17 or newer, Node.js 18 or newer, and an Aiven account.

1. **Create the database.** On console.aiven.io create a MySQL service on the Free plan. Copy the host, port, user and password.
2. **Start the backend.** In the `backend` folder, set `SPRING_PROFILES_ACTIVE=cloud`, `DB_URL=jdbc:mysql://HOST:PORT/defaultdb?sslMode=REQUIRED`, `DB_USER` and `DB_PASSWORD`, then run `.\mvnw.cmd spring-boot:run` (Windows) or `./mvnw spring-boot:run`. The first start creates all tables and demo data.

3. **Start the frontend.** In `frontend` run `npm install` then `npm run dev`, and open the printed link.
4. **Optional: recommender.** In `python-service` install the requirements and run `uvicorn app:app --port 8000`.

Demo accounts (password `demo123`): `hr@skillsync.ai`, `manager@skillsync.ai`, `employee@skillsync.ai`.

13. Configuration Reference

Variable	Default	Meaning
DB_URL, DB_USER, DB_PASSWORD	local MySQL	Database connection.
JWT_SECRET	placeholder	Signing key, at least 32 characters. Change it for real use.
CORS_ORIGIN	http://localhost:5173	Allowed frontend origin.
SEED_DEMO	true	Create demo accounts when the users table is empty.
ALLOW_ADMIN_SIGNUP	false	Let public sign-up create HR accounts.
VITE_API_URL	http://localhost:8080	Backend address used by the frontend.

14. Current Limitations

- Login, registration and accounts are stored in the database. The dashboard figures, employee list, courses and skill analysis still use sample data and are not yet read from the database.
- Password reset does not send email yet.
- The free Aiven database can be powered off after inactivity and must be switched back on from the console.
- The Java backend has not yet been run end to end by the author of this document; report any start-up errors.
- There are no automated tests yet.

15. Future Enhancements

- Read employees, skills, job roles and courses from MySQL and add create, edit and delete screens.
- Skill assessments with questions and manager evaluations.
- Connect the Python recommender to the Spring Boot API and store results in the recommendations table.
- Email delivery for password reset and training reminders.
- Exportable reports (PDF and Excel) and audit logs.
- Unit and integration tests, and containerized deployment.

16. Conclusion

SkillSync AI provides a working foundation for skill-gap management: secure role-based login backed by a hosted database, a polished analytics interface, a clear data model and a recommendation service. Connecting the remaining dashboard data to the database is the main step toward a production-ready system.